NAME : M.GURU BHAGAVAN

REG NO : 192312501

FIND-S Algorithm

Finding the Most Specific Hypothesis from Training Data

Five Problems with Aim, Algorithm, Program, Input and Output

Introduction

FIND-S is a concept-learning algorithm from machine learning. Starting from the most specific hypothesis, it scans the training examples and generalises the hypothesis only as much as needed to cover every positive example. Negative examples are ignored. The result is the most specific hypothesis in the hypothesis space that is consistent with all positive training examples.

Hypothesis notation: each attribute is either a specific value (e.g. Sunny), "?" meaning any value is acceptable, or "0" meaning no value is acceptable.

Limitations: FIND-S cannot detect noisy or inconsistent data, ignores negative examples, and returns only one hypothesis even when several are consistent. It assumes the target concept is a conjunction of attribute constraints.

General FIND-S Algorithm

1. Initialise h to the most specific hypothesis in H: h = <0, 0, 0, ..., 0> (one 0 for each attribute).
2. For each training example x:
    a. If x is a negative example, ignore it (h is not changed).
    b. If x is a positive example, then for each attribute a_i in h:
        i. If the constraint a_i is satisfied by x, do nothing.
        ii. Otherwise, replace a_i in h by the next more general constraint satisfied by x (a specific value becomes "?").
3. Output the hypothesis h.

Problem 1: EnjoySport Prediction

Aim

To implement the FIND-S algorithm to find the most specific hypothesis that describes the weather conditions under which a person enjoys a water sport.

Problem Statement

Given weather observations (Sky, AirTemp, Humidity, Wind, Water, Forecast) and whether the person enjoyed the sport (Yes/No), find the most specific hypothesis consistent with all positive examples.

Algorithm

Step 1: Initialise h = <0, 0, ..., 0>.

Step 2: Read each training example; if its target is Yes, treat it as positive, otherwise skip it.

Step 3: For the first positive example, copy its attribute values into h.

Step 4: For every later positive example, compare each attribute with h; if the values differ, replace that attribute in h with "?".

Step 5: After all examples are processed, print h as the final most specific hypothesis.

Input (Training Data)

Sky	AirTemp	Humidity	Wind	Water	Forecast	Target
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Program

```
# FIND-S Algorithm - Problem 1: EnjoySport

def find_s(attributes, data):
    """Return the most specific hypothesis consistent with the positive examples."""
    n = len(attributes)
    hypothesis = ['0'] * n            # Step 1: most specific hypothesis
    first positive = True


    print("Initial Hypothesis :", hypothesis)
    print("-" * 70)


    for i, row in enumerate(data, start=1):
        *features, label = row
        if label.lower() == 'yes':    # Step 2: consider positive examples only
            if first_positive:
                hypothesis = list(features)       # copy the first positive example
                first_positive = False
            else:
                for j in range(n):
                    if hypothesis[j] != features[j]:
                        hypothesis[j] = '?'       # generalise the mismatching attribute
            print(f"Example {i} (Positive) -> h = {hypothesis}")
        else:
            print(f"Example {i} (Negative) -> ignored, h = {hypothesis}")
    return hypothesis



attributes = ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
```

```
training_data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High',   'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High',   'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High',   'Strong', 'Cool', 'Change', 'Yes'],
]

print("Training Data:")
print(attributes + ['Target'])
for row in training data:
    print(row)
print()

final h = find s(attributes, training data)

print("-" * 70)
print("Final Most Specific Hypothesis:", final_h)
```

Output

```
Training Data:
['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast', 'Target']
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No']
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

Initial Hypothesis : ['0', '0', '0', '0', '0', '0']
----------------------------------------------------------------------
Example 1 (Positive) -> h = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
Example 2 (Positive) -> h = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
Example 3 (Negative) -> ignored, h = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
Example 4 (Positive) -> h = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
----------------------------------------------------------------------
Final Most Specific Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Result: The most specific hypothesis is ['Sunny', 'Warm', '?', 'Strong', '?', '?'].

Problem 2: Customer Buys a Computer

Aim

To use FIND-S to learn the most specific description of customers who buy a computer, based on Age, Income, Student status and Credit Rating.

Problem Statement

A shop records customer attributes and whether each customer purchased a computer. Find the most specific hypothesis that covers all buyers.

Algorithm

Step 1: Initialise h = <0, 0, ..., 0>.

Step 2: Read each training example; if its target is Yes, treat it as positive, otherwise skip it.

Step 3: For the first positive example, copy its attribute values into h.

Step 4: For every later positive example, compare each attribute with h; if the values differ, replace that attribute in h with "?".

Step 5: After all examples are processed, print h as the final most specific hypothesis.

Input (Training Data)

Age	Income	Student	CreditRating	Target
Youth	High	No	Fair	No
Middle	High	No	Fair	Yes
Middle	Medium	Yes	Excellent	Yes
Middle	Low	Yes	Excellent	Yes
Senior	Medium	No	Fair	No
Middle	Medium	Yes	Fair	Yes

Program

```
# FIND-S Algorithm - Problem 2: Customer Buys a Computer

def find_s(attributes, data):
    """Return the most specific hypothesis consistent with the positive examples."""
    n = len(attributes)
    hypothesis = ['0'] * n            # Step 1: most specific hypothesis
    first positive = True

    print("Initial Hypothesis :", hypothesis)
    print("-" * 70)

    for i, row in enumerate(data, start=1):
        *features, label = row
        if label.lower() == 'yes':    # Step 2: consider positive examples only
            if first positive:
                hypothesis = list(features)       # copy the first positive example
                first_positive = False
            else:
                for j in range(n):
                    if hypothesis[j] != features[j]:
                        hypothesis[j] = '?'       # generalise the mismatching attribute
            print(f"Example {i} (Positive) -> h = {hypothesis}")
        else:
            print(f"Example {i} (Negative) -> ignored, h = {hypothesis}")
    return hypothesis
```

```
attributes = ['Age', 'Income', 'Student', 'CreditRating']

training_data = [
    ['Youth',  'High',   'No',  'Fair',      'No'],
    ['Middle', 'High',   'No',  'Fair',      'Yes'],
    ['Middle', 'Medium', 'Yes', 'Excellent', 'Yes'],
    ['Middle', 'Low',    'Yes', 'Excellent', 'Yes'],
    ['Senior', 'Medium', 'No',  'Fair',      'No'],
    ['Middle', 'Medium', 'Yes', 'Fair',      'Yes'],
]

print("Training Data:")
print(attributes + ['Target'])
for row in training_data:
    print(row)
print()

final_h = find_s(attributes, training_data)

print("-" * 70)
print("Final Most Specific Hypothesis:", final_h)
```

Output

```
Training Data:
['Age', 'Income', 'Student', 'CreditRating', 'Target']
['Youth', 'High', 'No', 'Fair', 'No']
['Middle', 'High', 'No', 'Fair', 'Yes']
['Middle', 'Medium', 'Yes', 'Excellent', 'Yes']
['Middle', 'Low', 'Yes', 'Excellent', 'Yes']
['Senior', 'Medium', 'No', 'Fair', 'No']
['Middle', 'Medium', 'Yes', 'Fair', 'Yes']

Initial Hypothesis : ['0', '0', '0', '0']
----------------------------------------------------------------------
Example 1 (Negative) -> ignored, h = ['0', '0', '0', '0']
Example 2 (Positive) -> h = ['Middle', 'High', 'No', 'Fair']
Example 3 (Positive) -> h = ['Middle', '?', '?', '?']
Example 4 (Positive) -> h = ['Middle', '?', '?', '?']
Example 5 (Negative) -> ignored, h = ['Middle', '?', '?', '?']
Example 6 (Positive) -> h = ['Middle', '?', '?', '?']
----------------------------------------------------------------------
Final Most Specific Hypothesis: ['Middle', '?', '?', '?']
```

Result: The most specific hypothesis is ['Middle', '?', '?', '?'].

Problem 3: Edible Mushroom Classification

Aim

To apply FIND-S to identify the most specific set of features shared by all edible mushrooms in the training set.

Problem Statement

Mushrooms are described by Cap-Shape, Cap-Color, Odor, Gill-Size and Habitat, and labelled edible (Yes) or poisonous (No). Find the most specific hypothesis for edible mushrooms.

Algorithm

Step 1: Initialise h = <0, 0, ..., 0>.

Step 2: Read each training example; if its target is Yes, treat it as positive, otherwise skip it.

Step 3: For the first positive example, copy its attribute values into h.

Step 4: For every later positive example, compare each attribute with h; if the values differ, replace that attribute in h with "?".

Step 5: After all examples are processed, print h as the final most specific hypothesis.

Input (Training Data)

Cap-Shape	Cap-Color	Odor	Gill-Size	Habitat	Target
Convex	Brown	None	Broad	Woods	Yes
Flat	Yellow	Foul	Narrow	Grass	No
Convex	Brown	None	Broad	Woods	Yes
Convex	Brown	None	Broad	Meadows	Yes
Bell	White	Pungent	Narrow	Urban	No
Convex	Brown	None	Narrow	Woods	Yes

Program

```
# FIND-S Algorithm - Problem 3: Edible Mushroom Classification

def find_s(attributes, data):
    """Return the most specific hypothesis consistent with the positive examples."""
    n = len(attributes)
    hypothesis = ['0'] * n            # Step 1: most specific hypothesis
    first positive = True

    print("Initial Hypothesis :", hypothesis)
    print("-" * 70)

    for i, row in enumerate(data, start=1):
        *features, label = row
        if label.lower() == 'yes':    # Step 2: consider positive examples only
            if first positive:
                hypothesis = list(features)       # copy the first positive example
                first_positive = False
            else:
                for j in range(n):
                    if hypothesis[j] != features[j]:
                        hypothesis[j] = '?'       # generalise the mismatching attribute
            print(f"Example {i} (Positive) -> h = {hypothesis}")
        else:
            print(f"Example {i} (Negative) -> ignored, h = {hypothesis}")
    return hypothesis
```

```
attributes = ['Cap-Shape', 'Cap-Color', 'Odor', 'Gill-Size', 'Habitat']

training_data = [
    ['Convex', 'Brown',  'None',  'Broad',  'Woods',   'Yes'],
    ['Flat',   'Yellow', 'Foul',  'Narrow', 'Grass',   'No'],
    ['Convex', 'Brown',  'None',  'Broad',  'Woods',   'Yes'],
    ['Convex', 'Brown',  'None',  'Broad',  'Meadows', 'Yes'],
    ['Bell',   'White',  'Pungent','Narrow','Urban',   'No'],
    ['Convex', 'Brown',  'None',  'Narrow', 'Woods',   'Yes'],
]

print("Training Data:")
print(attributes + ['Target'])
for row in training_data:
    print(row)
print()

final_h = find_s(attributes, training_data)

print("-" * 70)
print("Final Most Specific Hypothesis:", final_h)
```

Output

```
Training Data:
['Cap-Shape', 'Cap-Color', 'Odor', 'Gill-Size', 'Habitat', 'Target']
['Convex', 'Brown', 'None', 'Broad', 'Woods', 'Yes']
['Flat', 'Yellow', 'Foul', 'Narrow', 'Grass', 'No']
['Convex', 'Brown', 'None', 'Broad', 'Woods', 'Yes']
['Convex', 'Brown', 'None', 'Broad', 'Meadows', 'Yes']
['Bell', 'White', 'Pungent', 'Narrow', 'Urban', 'No']
['Convex', 'Brown', 'None', 'Narrow', 'Woods', 'Yes']

Initial Hypothesis : ['0', '0', '0', '0', '0']
----------------------------------------------------------------------
Example 1 (Positive) -> h = ['Convex', 'Brown', 'None', 'Broad', 'Woods']
Example 2 (Negative) -> ignored, h = ['Convex', 'Brown', 'None', 'Broad', 'Woods']
Example 3 (Positive) -> h = ['Convex', 'Brown', 'None', 'Broad', 'Woods']
Example 4 (Positive) -> h = ['Convex', 'Brown', 'None', 'Broad', '?']
Example 5 (Negative) -> ignored, h = ['Convex', 'Brown', 'None', 'Broad', '?']
Example 6 (Positive) -> h = ['Convex', 'Brown', 'None', '?', '?']
----------------------------------------------------------------------
Final Most Specific Hypothesis: ['Convex', 'Brown', 'None', '?', '?']
```

Result: The most specific hypothesis is ['Convex', 'Brown', 'None', '?', '?'].

Problem 4: Flu Diagnosis

Aim

To implement FIND-S to derive the most specific combination of symptoms that characterises patients diagnosed with flu.

Problem Statement

Patient records contain Fever, Cough, Headache, Body Pain and Fatigue, with a diagnosis of Flu (Yes/No). Find the most specific hypothesis for positive flu cases.

Algorithm

Step 1: Initialise h = <0, 0, ..., 0>.

Step 2: Read each training example; if its target is Yes, treat it as positive, otherwise skip it.

Step 3: For the first positive example, copy its attribute values into h.

Step 4: For every later positive example, compare each attribute with h; if the values differ, replace that attribute in h with "?".

Step 5: After all examples are processed, print h as the final most specific hypothesis.

Input (Training Data)

Fever	Cough	Headache	BodyPain	Fatigue	Target
High	Yes	Yes	Yes	Yes	Yes
Normal	No	No	No	No	No
High	Yes	Yes	No	Yes	Yes
High	Yes	No	Yes	Yes	Yes
Normal	Yes	No	No	Yes	No

Program

```
# FIND-S Algorithm - Problem 4: Flu Diagnosis

def find_s(attributes, data):
    """Return the most specific hypothesis consistent with the positive examples."""
    n = len(attributes)
    hypothesis = ['0'] * n            # Step 1: most specific hypothesis
    first positive = True

    print("Initial Hypothesis :", hypothesis)
    print("-" * 70)

    for i, row in enumerate(data, start=1):
        *features, label = row
        if label.lower() == 'yes':    # Step 2: consider positive examples only
            if first positive:
                hypothesis = list(features)       # copy the first positive example
                first_positive = False
            else:
                for j in range(n):
                    if hypothesis[j] != features[j]:
                        hypothesis[j] = '?'       # generalise the mismatching attribute
            print(f"Example {i} (Positive) -> h = {hypothesis}")
        else:
            print(f"Example {i} (Negative) -> ignored, h = {hypothesis}")
    return hypothesis


attributes = ['Fever', 'Cough', 'Headache', 'BodyPain', 'Fatigue']
```

```
training_data = [
    ['High',   'Yes', 'Yes', 'Yes', 'Yes', 'Yes'],
    ['Normal', 'No',  'No',  'No',  'No',  'No'],
    ['High',   'Yes', 'Yes', 'No',  'Yes', 'Yes'],
    ['High',   'Yes', 'No',  'Yes', 'Yes', 'Yes'],
    ['Normal', 'Yes', 'No',  'No',  'Yes', 'No'],
]

print("Training Data:")
print(attributes + ['Target'])
for row in training_data:
    print(row)
print()

final_h = find_s(attributes, training_data)

print("-" * 70)
print("Final Most Specific Hypothesis:", final_h)
```

Output

```
Training Data:
['Fever', 'Cough', 'Headache', 'BodyPain', 'Fatigue', 'Target']
['High', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes']
['Normal', 'No', 'No', 'No', 'No', 'No']
['High', 'Yes', 'Yes', 'No', 'Yes', 'Yes']
['High', 'Yes', 'No', 'Yes', 'Yes', 'Yes']
['Normal', 'Yes', 'No', 'No', 'Yes', 'No']

Initial Hypothesis : ['0', '0', '0', '0', '0']
----------------------------------------------------------------------
Example 1 (Positive) -> h = ['High', 'Yes', 'Yes', 'Yes', 'Yes']
Example 2 (Negative) -> ignored, h = ['High', 'Yes', 'Yes', 'Yes', 'Yes']
Example 3 (Positive) -> h = ['High', 'Yes', 'Yes', '?', 'Yes']
Example 4 (Positive) -> h = ['High', 'Yes', '?', '?', 'Yes']
Example 5 (Negative) -> ignored, h = ['High', 'Yes', '?', '?', 'Yes']
----------------------------------------------------------------------
Final Most Specific Hypothesis: ['High', 'Yes', '?', '?', 'Yes']
```

Result: The most specific hypothesis is ['High', 'Yes', '?', '?', 'Yes'].

Problem 5: Student Placement Prediction

Aim

To use FIND-S to find the most specific profile of students who get placed, based on academic and skill attributes.

Problem Statement

Student records contain CGPA, Internships, Communication skill, Projects and Backlogs, with the result Placed (Yes/No). Find the most specific hypothesis for placed students.

Algorithm

Step 1: Initialise h = <0, 0, ..., 0>.

Step 2: Read each training example; if its target is Yes, treat it as positive, otherwise skip it.

Step 3: For the first positive example, copy its attribute values into h.

Step 4: For every later positive example, compare each attribute with h; if the values differ, replace that attribute in h with "?".

Step 5: After all examples are processed, print h as the final most specific hypothesis.

Input (Training Data)

CGPA	Internships	Communication	Projects	Backlogs	Target
High	Yes	Good	Many	None	Yes
Low	No	Poor	Few	Some	No
High	Yes	Good	Few	None	Yes
Medium	No	Good	Many	Some	No
High	Yes	Good	Many	None	Yes

Program

```
# FIND-S Algorithm - Problem 5: Student Placement

def find_s(attributes, data):
    """Return the most specific hypothesis consistent with the positive examples."""
    n = len(attributes)
    hypothesis = ['0'] * n            # Step 1: most specific hypothesis
    first positive = True

    print("Initial Hypothesis :", hypothesis)
    print("-" * 70)

    for i, row in enumerate(data, start=1):
        *features, label = row
        if label.lower() == 'yes':    # Step 2: consider positive examples only
            if first positive:
                hypothesis = list(features)       # copy the first positive example
                first_positive = False
            else:
                for j in range(n):
                    if hypothesis[j] != features[j]:
                        hypothesis[j] = '?'       # generalise the mismatching attribute
            print(f"Example {i} (Positive) -> h = {hypothesis}")
        else:
            print(f"Example {i} (Negative) -> ignored, h = {hypothesis}")
    return hypothesis


attributes = ['CGPA', 'Internships', 'Communication', 'Projects', 'Backlogs']
```

```
training_data = [
    ['High',   'Yes', 'Good', 'Many', 'None', 'Yes'],
    ['Low',    'No',  'Poor', 'Few',  'Some', 'No'],
    ['High',   'Yes', 'Good', 'Few',  'None', 'Yes'],
    ['Medium', 'No',  'Good', 'Many', 'Some', 'No'],
    ['High',   'Yes', 'Good', 'Many', 'None', 'Yes'],
]

print("Training Data:")
print(attributes + ['Target'])
for row in training_data:
    print(row)
print()

final_h = find_s(attributes, training_data)

print("-" * 70)
print("Final Most Specific Hypothesis:", final_h)
```

Output

```
Training Data:
['CGPA', 'Internships', 'Communication', 'Projects', 'Backlogs', 'Target']
['High', 'Yes', 'Good', 'Many', 'None', 'Yes']
['Low', 'No', 'Poor', 'Few', 'Some', 'No']
['High', 'Yes', 'Good', 'Few', 'None', 'Yes']
['Medium', 'No', 'Good', 'Many', 'Some', 'No']
['High', 'Yes', 'Good', 'Many', 'None', 'Yes']

Initial Hypothesis : ['0', '0', '0', '0', '0']
----------------------------------------------------------------------
Example 1 (Positive) -> h = ['High', 'Yes', 'Good', 'Many', 'None']
Example 2 (Negative) -> ignored, h = ['High', 'Yes', 'Good', 'Many', 'None']
Example 3 (Positive) -> h = ['High', 'Yes', 'Good', '?', 'None']
Example 4 (Negative) -> ignored, h = ['High', 'Yes', 'Good', '?', 'None']
Example 5 (Positive) -> h = ['High', 'Yes', 'Good', '?', 'None']
----------------------------------------------------------------------
Final Most Specific Hypothesis: ['High', 'Yes', 'Good', '?', 'None']
```

Result: The most specific hypothesis is ['High', 'Yes', 'Good', '?', 'None'].